

Instituto Tecnológico de Milpa Alta II

Ingeniería en Sistemas Computacionales

Mockups y Casos de Uso

Materia: Fundamentos de Ingeniería de Software

Profesor: Mtro. Roldán Aquino Segura

Equipo: Ingeniebríos

- Marco Antonio Aguilar Guerrero
- César Meza Corella
- Edgar Salazar López
- Misael Silva Fragoso

Fecha: 23 de Abril de 2026

Gestión de Inventario - Papelería

Organiza productos faltantes y pedidos de clientes para mantener tu stock actualizado

Gestión de Inventario - Papelería

Organiza productos faltantes y pedidos de clientes para mantener tu stock actualizado



Productos Faltantes



Pedidos Nuevos



Exportar a PDF

Productos Faltantes

+ Agregar Producto

Cuadernos A4 Cuadriculados

ALTA

Sin stock disponible.

Bolígrafos Azules (Caja x50)

MEDIA

Quedan 2 unidades.

Gomas de borrar

BAJA

En stock, pero bajo.

Introducción: Mockups y Casos de Uso

En este apartado revisaremos a detalle la estructura de nuestro proyecto y cómo se realizaría desde un estado inicial a un estado final.

Dentro de este proyecto usaremos herramientas como **Canva** para la creación de Mockups (los cuales también codificamos y anexamos aquí mismo de forma interactiva) además de agregar sus casos de uso.

Para la planificación del proyecto incluiremos el cronograma de actividades y un listado de tareas usando la herramienta **Jira**.

Por último especificaremos la estructura de todo nuestro repositorio para un fácil uso de este.

Elementos Principales de la Herramienta

- **Productos Faltantes:** Sección dedicada a listar artículos agotados o por agotarse. Permite la acción "Agregar Producto". En su estado inicial o cuando el inventario está completo, muestra el mensaje: "No hay productos faltantes registrados".
- **Pedidos Nuevos:** Área destinada a registrar encargos de clientes.
- **Exportar a PDF:** Funcionalidad clave que permite guardar y compartir el reporte del inventario actual.

Cómo construimos el mockup

Para desarrollar esta interfaz, tomamos una serie de decisiones técnicas y de diseño orientadas a la simplicidad y la usabilidad:

- **React con useState y useEffect:** Elegimos React por su capacidad para manejar interfaces dinámicas. Utilizamos useState para crear listas interactivas y useEffect para reaccionar a los cambios de estado.
- **Separación entre productos y pedidos:** Hemos dividido la información en dos categorías lógicas: "Productos faltantes" para venta regular y "Pedidos nuevos" para encargos de clientes, evitando confusión visual.
- **Pestañas para cambiar de vista:** Implementamos un sistema de pestañas (Tabs) tipo botón ('pill') que permite concentrarse en una sola tarea manteniendo la interfaz minimalista.
- **localStorage para persistencia:** Conectamos el estado de la app con el navegador. Así, la información perdura de forma local sin necesidad de configurar una base de datos externa.
- **Botón de exportación:** Incorporamos una acción global destacada en verde para exportar los datos, una función vital en el entorno comercial.
- **Iconos y jerarquía visual:** Utilizamos iconos representativos, fondos claros, bordes suaves y tarjetas estructuradas para guiar la vista hacia el contenido importante.

Estructura visual del mockup

- **Encabezado:** Título grande y claro acompañado de un subtítulo descriptivo que contextualiza rápidamente la herramienta.

- **Navegación por Pestañas:** Diseño en formato 'pill' interactivo, donde el color azul vivo indica la sección activa frente al fondo blanco inactivo.
- **Panel de Productos Faltantes:** Tarjeta principal expansiva que contiene listas de inventario o un estado vacío claro y amigable.
- **Panel de Pedidos Nuevos:** Vista alternativa estructurada para listar los requerimientos específicos solicitados por clientes directos.
- **Acción de Exportar:** Botón superior derecho contrastante (verde) para generar reportes en PDF de manera accesible e inmediata.

Flujo de uso

1. **Revisión:** El encargado inspecciona el local y abre la pestaña de faltantes.
2. **Registro:** Anota los productos que se agotan y clasifica su urgencia.
3. **Atención:** En la segunda pestaña, toma nota de los encargos de clientes.
4. **Exportación:** Al final del día, pulsa "Exportar a PDF" para enviar el listado al proveedor.

Ejemplo de productos y prioridades

Cuando el inventario está poblado, las tarjetas internas lucen así, destacando la urgencia de reposición mediante insignias de texto (badges):

- Cuadernos A4 Cuadriculados - **[ALTA]**: Sin stock disponible.
- Bolígrafos Azules (Caja x50) - **[MEDIA]**: Quedan 2 unidades.
- Gomas de borrar - **[BAJA]**: En stock, pero bajo.

Código principal

```
import { useState, useEffect } from "react";
import { Tabs, TabsContent, TabsList, TabsTrigger } from "@radix-ui/react-tabs";
import { MissingProducts } from "../components/MissingProducts";
import { NewOrders } from "../components/NewOrders";
import { ExportButton } from "../components/ExportButton";
import { Package, ShoppingCart } from "lucide-react";

export interface MissingProduct {
  id: string;
  name: string;
  inStock: boolean;
  urgency: "low" | "medium" | "high";
  photo?: string;
}
```

```

export interface NewOrder {
  id: string;
  productName: string;
  requestCount: number;
}

export default function App() {
  const [missingProducts, setMissingProducts] = useState<MissingProduct[]>([]);
  const [newOrders, setNewOrders] = useState<NewOrder[]>([]);

  // Load data from localStorage on mount
  useEffect(() => {
    const savedMissingProducts = localStorage.getItem("missingProducts");
    const savedNewOrders = localStorage.getItem("newOrders");

    if (savedMissingProducts) {
      setMissingProducts(JSON.parse(savedMissingProducts));
    }
    if (savedNewOrders) {
      setNewOrders(JSON.parse(savedNewOrders));
    }
  }, []);

  // Save to localStorage whenever data changes
  useEffect(() => {
    localStorage.setItem("missingProducts", JSON.stringify(missingProducts));
  }, [missingProducts]);

  useEffect(() => {
    localStorage.setItem("newOrders", JSON.stringify(newOrders));
  }, [newOrders]);

  return (
    <div className="min-h-screen bg-gray-50 p-6">
      <div className="max-w-6xl mx-auto">
        <div className="mb-8">
          <h1 className="text-4xl font-bold text-gray-900 mb-2">
            Gestión de Inventario - Papelería
          </h1>
          <p className="text-gray-600">
            Organiza productos faltantes y pedidos de clientes para mantener tu stock act
          </p>
        </div>

        <Tabs defaultValue="missing" className="w-full">
          <div className="flex justify-between items-center mb-6">
            <TabsList className="inline-flex h-12 items-center justify-center rou">
              <TabsTrigger value="missing" className="inline-flex items-center justify-ce
                <Package className="mr-2 h-4 w-4" /> Productos Faltantes
              </TabsTrigger>
              <TabsTrigger value="orders" className="inline-flex items-center justify-cen
                <ShoppingCart className="mr-2 h-4 w-4" /> Pedidos Nuevos
              </TabsTrigger>
            </TabsList>
            <ExportButton missingProducts={missingProducts} newOrders={newOrders} />
          </div>

          <TabsContent value="missing" className="mt-0">

```

```

        <MissingProducts products={missingProducts} setProducts={setMissingProducts}
        </TabsContent>

        <TabsContent value="orders" className="mt-0">
          <NewOrders orders={newOrders} setOrders={setNewOrders} />
        </TabsContent>
      </Tabs>
    </div>
  </div>
);
}

```

Explicación paso a paso del código

Para comprender mejor cómo funciona el sistema internamente, vamos a traducir el código paso a paso:

1. **Las importaciones iniciales:** Al principio del documento, traemos las herramientas que necesitamos de React (para la memoria o estado) y librerías externas para estructurar visualmente el sistema de pestañas e iconos.
2. **Definición de las reglas (Interfaces):** Establecemos el "molde" exacto para nuestra información. Un "Producto Faltante" necesita un ID, nombre, disponibilidad y nivel de urgencia. Un "Pedido Nuevo" requiere ID, nombre y la cantidad de veces solicitado.
3. **Creación de la memoria principal (useState):** Iniciamos la aplicación creando dos contenedores dinámicos en la pantalla: uno para listar los faltantes y otro para los pedidos.
4. **Recuperación de datos (Primer useEffect):** Cuando la página carga, el sistema revisa inmediatamente el localStorage del navegador. Si encuentra información de sesiones previas, la carga automáticamente para no perder nuestro trabajo anterior.
5. **Guardado automático (Segundos useEffect):** Le instruimos a la aplicación que cada vez que modifiquemos las listas (agregando, editando o eliminando), realice una copia de seguridad invisible e instantánea en el navegador.
6. **La estructura visual (Bloque return):** Finalmente, el código construye la interfaz gráfica que vemos. Dibuja el gran título, despliega los botones de navegación y el botón de exportación. Dependiendo de la pestaña en la que hagamos clic, mostrará el panel correspondiente a faltantes o pedidos en la zona inferior.

Herramientas que utilizamos y cómo fue nuestro proceso

Para sacar adelante este proyecto, nuestro equipo se apoyó en varias herramientas tecnológicas. Para el diseño visual rápido y los casos de uso utilizamos **Canva**. En cuanto

al desarrollo del mockup funcional, decidimos usar HTML, CSS y React para darle esa sensación interactiva y fluida a la interfaz. Para no perder el código y tener todo bien respaldado, usamos Git y lo subimos a nuestro repositorio en GitHub. Además, para la planificación y no perdernos con las fechas, armamos nuestro listado de tareas en **Jira**.

¿Cómo fue el proceso de elaboración?

Todo empezó cuando analizamos la situación de la papelería y notamos el problema que tenían con el descontrol de su mercancía. Después de la entrevista, nos sentamos a pensar cómo solucionarlo y se nos ocurrió separar la pantalla en dos vistas súper claras: lo que ya no hay en tienda y lo que la gente deja encargado. A partir de ahí, diseñamos en Canva y luego empezamos a programar la vista. Le metimos etiquetas de colores (alta, media, baja) para que el dueño sepa qué comprar con urgencia. Una vez que quedó el diseño, juntamos todos nuestros reportes y subimos todo a GitHub para armar esta entrega final.

Planificación de nuestro proyecto

Cronograma de actividades

Como nuestro tiempo de trabajo ha sido de semana y media, nos organizamos dividiendo las labores en días para no dejar todo a última hora:

- **Días 1 a 2 (Investigación):** Fuimos a la papelería, hicimos la entrevista y detectamos cuál era el dolor principal del negocio.
- **Días 3 a 4 (Planeación):** Hicimos nuestras listas de tareas en Jira, calculamos tiempos y empezamos a bocear en Canva cómo queríamos que se viera la plataforma.
- **Días 5 a 8 (Desarrollo):** Fueron los días más pesados; el equipo se dedicó a programar la lógica del mockup interactivo y dejar los estilos visuales bien pulidos.
- **Días 9 a 10 (Cierre y Documentación):** Nos dedicamos a redactar los reportes en PDF, armar este documento final y subir todo el código a la nube para la entrega.

Nuestro listado de tareas en Jira

Así fue como dividimos las actividades específicas en Jira junto con los tiempos que calculamos para el equipo:

- **Tarea 1:** Preparar y realizar la entrevista de requerimientos (Aprox. 2 horas).
- **Tarea 2:** Analizar los datos y redactar el `Reporte_problema.pdf` (Aprox. 3 horas).
- **Tarea 3:** Diseñar los mockups y flujos de casos de uso en Canva (Aprox. 4 horas).
- **Tarea 4:** Programar todo el mockup interactivo en código HTML/React (Nos llevó unas 10 horas).

- **Tarea 5:** Hacer los commits, enlazar nuestro Git y subirlo a GitHub (Aprox. 2 horas).
- **Tarea 6:** Maquetar este documento HTML final para explicar todo el proyecto (Aprox. 3 horas).

Entrega de nuestro repositorio

Ya subimos todo nuestro código y archivos a GitHub para que lo puedan revisar a detalle. Les dejamos nuestro link oficial:

Enlace a nuestro repositorio: <https://github.com/YvnPretty/PAPELERIAxd>

Estructura de lo que estamos entregando

Para que sea súper fácil de revisar, dejamos nuestro repositorio organizado con estos tres archivos clave:

- **Reporte_entrevista_1.pdf:** Aquí están todas nuestras notas y hallazgos de la primera vez que fuimos a hablar con los de la papelería.
- **Reporte_problema.pdf:** Este es el análisis técnico donde explicamos por qué se les pierden los pedidos y cómo esta herramienta los va a ayudar a solucionarlo.
- **Diseño_de_la_solución.pdf**